

Introduction au Machine Learning

Guillaume Guex guillaume.guex@unil.ch

Le Machine Learning

Le **Machine Learning**, ou **Apprentissage Automatique** en français, est un domaine à mi-chemin entre la statistique et l'informatique.

Par rapport à la statistique, ce domaine met plus d'emphasis sur l'ordinateur pour approximer des fonctions complexes, et moins d'emphasis sur des démonstrations pour construire des intervalles de confiance ou des tests.

Le Machine Learning est le domaine à la base de l'**Intelligence Artificielle (IA)** moderne.

Pour [Mitchell, T. (1997). Machine Learning. McGraw Hill. p. 2] :

« A computer program is said to learn from **experience E** with respect to some class of **tasks T** and **performance measure P** if its **performance at tasks in T, as measured by P, improves with experience E.** »

« Dessine
une maison »



Données



Entrainement

« Dessine
une maison »



Les modèles donnant des résultats spectaculaires sont fréquemment **inaccessibles** au grand public, car ils nécessitent de **très nombreuses données** et une **grande puissance de calcul**.

Mais le ML reste utile à petite ou moyenne échelle !

Les algorithmes de ML sont **omniprésents** dans le monde actuel. Même si l'on ne compte pas les utiliser activement, **il est important de les comprendre** (un minimum).

Les méthodes

Nous avons vu que les méthodes de ML vont s'améliorer dans leur **Tâche T**, grâce à de l'**expérience E (les données)**, selon une **mesure de performance P**. Les **tâches** peuvent être variées :

- Classification.
- Traduction.
- Représentation de données.
- Génération de textes.
- Génération d'images.
- etc.

La mesure de performance P

Il s'agit de définir une **mesure de performance P** ou une **mesure de l'erreur** dans la tâche, afin de **mesurer l'amélioration** de la méthode.

- Si la tâche consiste à prédire une classe, cela peut-être le **pourcentage d'objets correctement classés** (vs une annotation humaine), que l'on nomme **exactitude**.
- Si la tâche consiste à prédire une valeur numérique, on peut calculer **l'erreur carrée moyenne (MSE)** entre les prédictions et des données annotées.

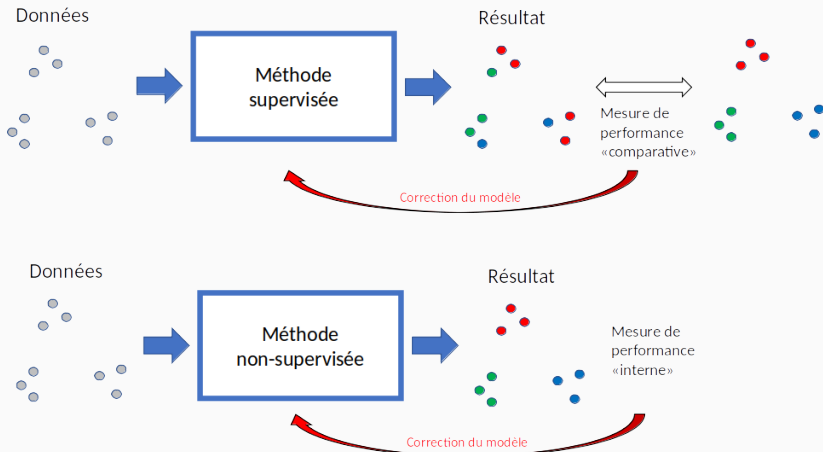
Parfois, cette mesure P peut-être plus compliquée à construire, comme lors d'une traduction automatique ou d'un chatbot.

On entraine souvent les modèles sur une **mesure de substitution (surrogate)**, plus facile à utiliser d'un point de vue mathématique.

Les **méthodes** de ML font partie de 2 groupes principaux :

- Les méthodes **supervisées** cherchent à prédire une (des) variable(s) existante(s) dans les données. On peut donc baser l'entraînement sur la **justesse des prédictions**.
- Les méthodes **non-supervisées** peuvent être prédictives (ou non), mais n'ont pas accès à la variable à prédire dans les données. Elles utilisent alors les **structures** présentes dans les données pour l'entraînement.

Méthodes supervisées et non-supervisées



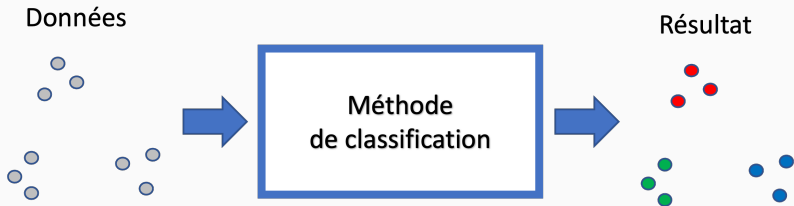
Il existe également d'autres types de méthodes :

- Les méthodes **semi-supervisées** ont accès à la variable à prédire sur **certaines données seulement**. Elles sont à mi-chemin entre les méthodes supervisées et non-supervisées.
- Les méthodes d'**apprentissage par renforcement** sont particulières, dans le sens qu'il n'y a pas réellement de jeu de données et que l'entraînement se base sur **une réponse (à la sortie de l'algorithme) donnée par un « environnement »**. (p.ex. IA de jeux vidéo)

Les méthodes de ML se divisent encore en deux catégories :

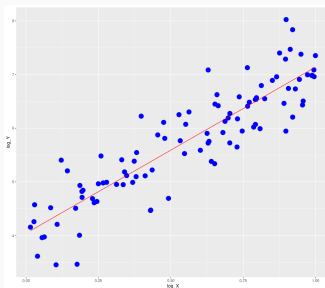
- Lorsque la variable à prédire est **catégorielle**, on parle de **classification**
- Lorsque la variable à prédire est **numérique**, on parle de **régression**.

Classification vs Régression



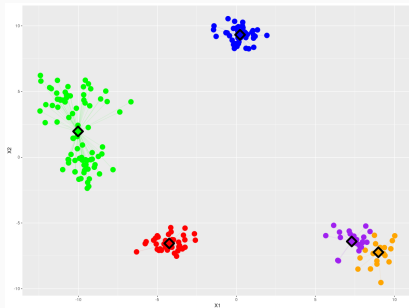
Exemple 1 : la régression linéaire

Type	Méthode de régression supervisée
Tâche T	Prédire le score d'une variable numérique Y d'un individu
Performance P	L'erreur quadratique moyenne $MSE = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$
Méthode	On suppose que $Y \sim b_0 + b_1X_1 + \dots + b_pX_p$ On trouve $b_0, \dots, b_p$ qui minimisent la MSE par calcul



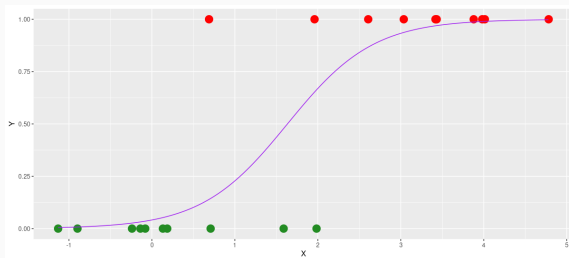
Exemple 2 : les k-moyennes

Type	Méthode de classification non-supervisée
Tâche T	Prédire la modalité d'une variable catégorielle Y d'un individu
Performance P	La variance inter-groupe.
Méthode	Algorithme itératif de type EM



Exemple 3 : la régression logistique

Type	Méthode de classification supervisée
Tâche T	Prédire la modalité d'une variable catégorielle Y d'un individu
Performance P	l'entropie relative
Méthode	On pose $P(Y = i) \sim \text{softmax}_i(b_0 + b_1X_1 + \dots + b_pX_p)$ On trouve la solution par algorithme itératif.



Mesures de performance d'une classification

L'exactitude

La mesure classique pour estimer la performance d'une classification est l'**exactitude (accuracy)** :

$$\text{exactitude} = \frac{\text{\#individus bien classés}}{\text{\#individus}}$$

Cependant, cette mesure peut être inadéquate. Un exemple via une **table de confusion** :

	Classé comme 0	Classé comme 1
Véritable 0	978	15
Véritable 1	5	2

$$\text{exactitude} = 980/1000 = 98\%.$$

Si ce tableau représente le résultat d'une recherche internet :

	Pas dans le résultat	Dans le résultat
Page non-pertinente	978	15
Page pertinente	5	2

La **précision** se définit comme :

$$\begin{aligned}\text{précision} &= P(\text{Page pertinente} | \text{Dans le résultat}) \\ &= 2/17 = 11.8\%\end{aligned}$$

Le nombre de page pertinentes dans le retour de la recherche est médiocre.

Le rappel

Si ce tableau représente un **test de dépistage d'une maladie** :

	Test négatif	Test positif
Sain	978	15
Malade	5	2

Le **rappel (recall)** se définit comme :

$$\begin{aligned}\text{rappel} &= P(\text{Test positif}|\text{Malade}) \\ &= 2/7 = 28.6\%\end{aligned}$$

Il y aura donc un nombre important de personnes malades non-dépistées.

Il existe également une mesure, appelée **mesure F_1** , qui combine précision et rappel :

$$F_1 = 2 \cdot \frac{\text{précision} \cdot \text{rappel}}{\text{précision} + \text{rappel}}$$

Cette mesure peut être généralisée avec la **mesure F_β** ($\beta > 0$) :

$$F_\beta = \frac{(1 + \beta^2) \cdot \text{précision} \cdot \text{rappel}}{(\beta^2 \cdot \text{précision}) + \text{rappel}}$$

	Classé comme 0	Classé comme 1
Véritable 0	a	b
Véritable 1	c	d

$$\text{exactitude} = \frac{a + d}{a + b + c + d}$$

$$\text{précision} = \frac{d}{b + d}$$

$$\text{rappel} = \frac{d}{c + d}$$

$$F_1 = 2 \cdot \frac{\text{précision} \cdot \text{rappel}}{\text{précision} + \text{rappel}}$$

Lorsque l'on a de **multiple classes** m , la précision et le rappel peuvent s'utiliser en faisant leur moyenne sur les classes :

$$\text{précision totale} = \frac{1}{m} \sum_{c=1}^m (\text{précision sur } c)$$

$$\text{rappel total} = \frac{1}{m} \sum_{c=1}^m (\text{rappel sur } c)$$

La généralisation

Un des enjeux du Machine Learning est d'arriver à ce que les méthodes **généralisent au-delà du jeu d'entraînement**.

Si le modèle utilisé est **complexe**, ce dernier va avoir de la peine à faire la distinction entre :

- Les **particularités** dues à l'aléatoire de l'échantillon.
- Les **régularités** réellement existantes dans la population.

On parle alors de phénomène de **sur-apprentissage** ou de **sur-paramétrisation**.

Famille de modèles : hyperparamètres

Supposons que l'on cherche à prédire la variable numérique Y avec la variable numérique X . On peut s'intéresser à la **famille de modèles linéaires** suivant :

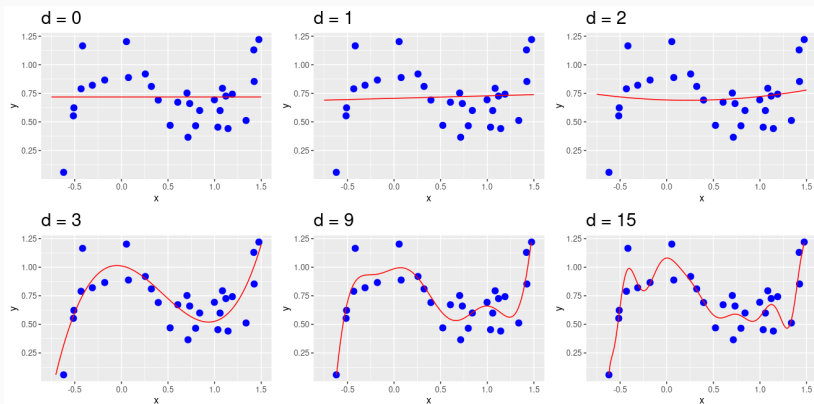
M(0)	$Y \sim b_0$
M(1)	$Y \sim b_0 + b_1X$
M(2)	$Y \sim b_0 + b_1X + b_2X^2$
M(3)	$Y \sim b_0 + b_1X + b_2X^2 + b_3X^3$
$\vdots$	$\vdots$

En réalité, on peut voir cette famille comme un seul modèle possédant un **hyperparamètre d** :

$$M(d) : Y \sim b_0 + b_1X + b_2X^2 + \dots + b_dX^d$$

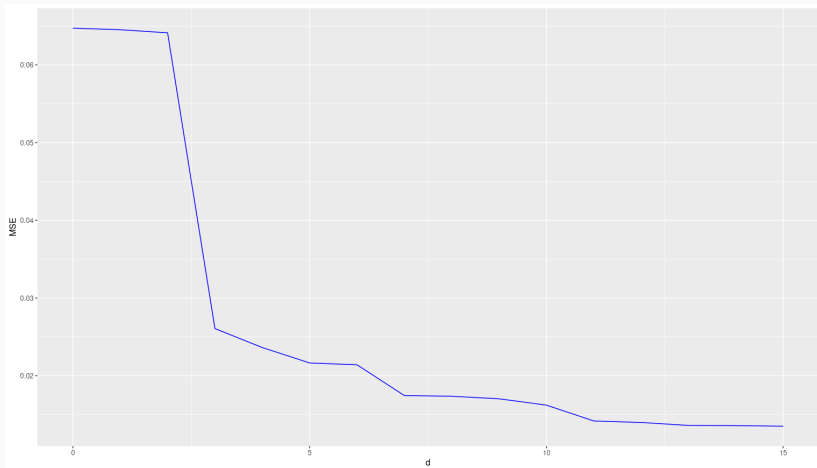
Famille de modèles : hyperparamètres

Avec un exemple de données, voici ce que le modèle va estimer en minimisant l'**erreur quadratique moyenne MSE** , en fonction de l'**hyperparamètre d** :



Famille de modèles : hyperparamètres

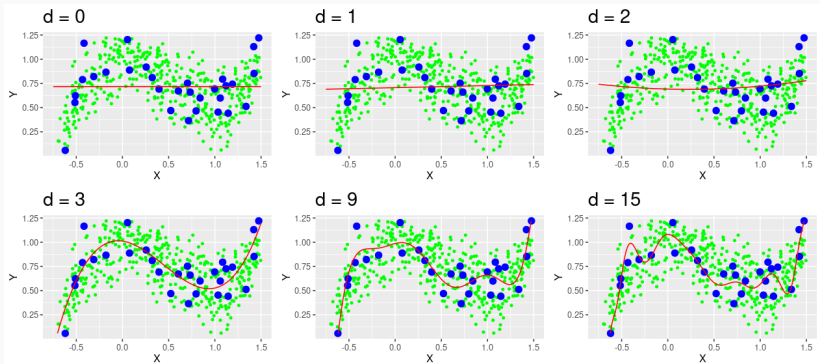
L'erreur MSE ne fait que **diminuer en fonction de d** :



Sur-apprentissage et sous-apprentissage

En réalité, ces données viennent du modèle génératif

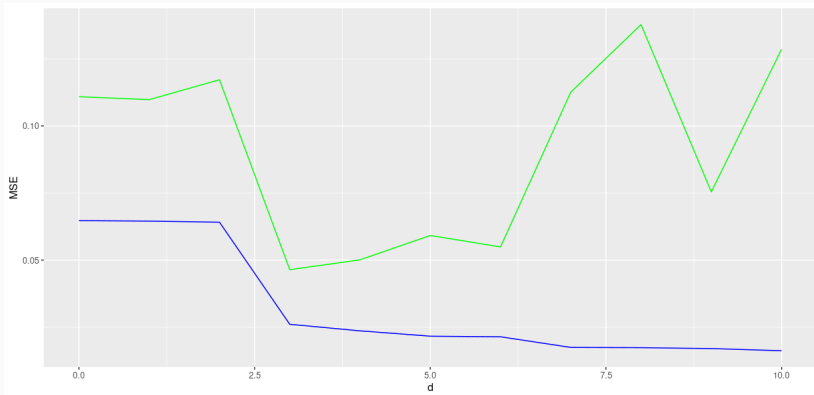
$$Y = X^3 - 1.5X^2 + 1 + \text{bruit}$$



Les modèles avec d trop élevés sont en **sur-apprentissage (overfitting)**, ceux avec un d trop petits en **sous-apprentissage (underfitting)**.

Famille de modèles : hyperparamètres

En regardant l'erreur MSE par rapport aux nouveaux points (en verts), on voit apparaître l'**hyperparamètre idéal** à $d = 3$:



Le phénomène de sous- et sur-apprentissage peut être observé en **séparant** notre jeu de données en deux groupes :

- Le jeu d'**entraînement** ($\sim 80\%$ des données) va être utilisé pour **entraîner** le modèle.
- Le jeu de **test** ($\sim 20\%$ des données) va être utilisé pour **estimer sa capacité de généralisation**.

Pour créer ces deux jeux de données, on tâchera d'**échantillonner** tout en **conservant la structure** du jeu de données initial (p.ex. même proportion d'individus dans chaque classe).

Le surapprentissage peut aussi se produire lorsque que l'on a de **nombreuses variables d'entrée par rapport au nombre d'individus**. On peut alors ajouter un terme de **régularisation** à la quantité à minimiser.

Par exemple, dans une régression linéaire, on va chercher à minimiser :

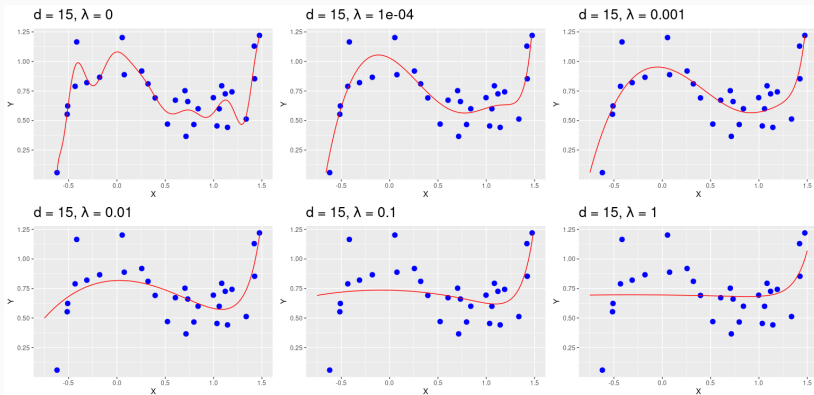
$$MSE + \lambda(b_0^2 + b_1^2 + \dots + b_p^2)$$

L'**hyperparamètre** $\lambda > 0$ va contrôler la complexité du modèle. Cette regression s'appelle la **régression Ridge**.

La **régression Lasso** utilise le même principe, mais la minimisation se fait sur $MSE + \lambda|b_0 + b_1 + \dots + b_p|$.

La régularisation

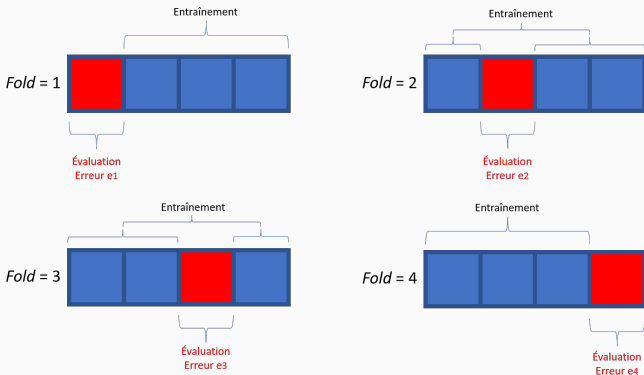
La complexité du modèle **diminue en augmentant λ** . Pour la régression Ridge, cela donne :



Cross-validation et sélection de modèles

La cross-validation

La **cross-validation avec k -folds** est une procédure qui permet d'avoir une **estimation robuste de l'erreur de généralisation**. Cela consiste à séparer notre jeu de données en k parties égales, puis de faire k séries d'entraînement/test :



$$\text{Erreur moyenne} = \frac{1}{4} (e_1 + e_2 + e_3 + e_4)$$

On peut s'en servir pour **sélectionner un modèle** parmi une collection, ou **ajuster des hyperparamètres**.

Par exemple, on se donne une régression Ridge avec $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$. Pour chacune de ces valeurs, on effectue une cross-validation puis on sélectionne le modèle avec la plus petite erreur de généralisation.

S'il existe **plusieurs hyperparamètres**, on peut soit effectuer une **recherche sur grille (toutes les possibilités des paramètres)**, soit une **recherche aléatoire**.

Attention, lorsqu'une sélection de modèle s'opère, l'erreur donnée par le modèle retenu est **trop optimiste**. **La procédure de sélection a été « ajustée » à tout le jeu de données !**

On doit donc séparer notre jeu de données en :

- Le **jeu d'entraînement** ($\sim 70\%$) permet d'entraîner les modèles.
- Le **jeu de validation** ($\sim 15\%$) permet de sélectionner le meilleur modèle.
- Le **jeu de test** ($\sim 15\%$) permet d'évaluer l'erreur de généralisation du meilleur modèle.

La cross-validation peut aussi être **imbriquée** (nested).